



week8

Sequence to Sequence Learning with Neural Networks

저자: Ilya Sutskever, Oriol Vinyals, Quoc V. Le (Google)

발행: arXiv (2014)

이 논문에서는 Sequence-to-Sequence (Seq2Seq) 모델을 소개하며, 특히 LSTM(Long Short-Term Memory) 기반의 인코더-디코더 구조를 사용하여 기계 번역(Machine Translation, MT)을 수행하는 방법을 제안.

핵심 기여

- 기존의 딥러닝 모델(DNN)은 고정된 입력/출력 차원을 다루지만, 이 논문은 이를 극복하여 **가변 길이 시퀀스를 변환할 수 있는 모델**을 제안함.
- **LSTM 기반 인코더-디코더 구조**를 사용하여 입력 시퀀스를 **고정된 벡터로 인코딩**한 뒤, 다시 이를 통해 출력 시퀀스를 생성하는 방법을 개발.
- **문장 순서를 뒤집는(trick)** 기법을 적용하여 성능을 개선함.
- WMT'14 **영어-프랑스어 기계 번역 데이터셋**에서 **BLEU 점수 34.8**을 달성하며, 기존 통계 기반 번역 시스템(SMT)을 능가.

Seq2Seq 모델 개요

(1) 기존 문제점

- DNN(Deep Neural Networks)은 강력한 모델이지만, 일반적으로 고정된 크기의 입력과 출력을 필요로 함.
- 번역, 음성 인식과 같은 문제는 시퀀스 데이터를 다루어야 하지만, 기존 DNN 모델은 이러한 유동적인 입력 길이를 다루기 어려움.

(2) Seq2Seq 모델의 핵심 아이디어

- **LSTM 기반 인코더(Encoder)**: 입력 문장을 차례로 읽고, 하나의 고정 길이 벡터(Vector)로 압축.
- **LSTM 기반 디코더(Decoder)**: 인코더가 생성한 벡터를 기반으로 새로운 문장을 생성.
- 입력과 출력의 길이가 다를 수 있어도 자연스럽게 처리 가능.

(3) 모델의 특이점

- **두 개의 LSTM 사용**: 하나는 입력을 인코딩하고, 다른 하나는 출력을 생성하는 구조.

- **입력 문장 순서 뒤집기(Reverse Sentence Order):** 입력 문장을 거꾸로 배치하면 번역 과정에서 단기 의존성(short-term dependency)이 강화되어 학습이 쉬워짐.
- **Beam Search 사용:** 여러 개의 후보 번역을 생성한 후 가장 가능성이 높은 문장을 선택하는 방식.

실험 및 결과

(1) WMT'14 영어-프랑스어 번역 실험

- 12M개의 문장(3억 개 이상의 단어)을 사용하여 학습.
- **단어 사전 크기:** 입력(16만 개), 출력(8만 개)
- **모델 구조:** 4층(Deep) LSTM, 1000차원 임베딩, 총 **3억8400만 개의 파라미터**
- **성능 평가:** BLEU 점수로 측정 (점수가 높을수록 번역 품질이 좋음)

모델	BLEU 점수
기존 SMT 시스템	33.3
Seq2Seq (단일 LSTM)	30.6
Seq2Seq (5개 LSTM 앙상블)	34.8
SMT + LSTM 재랭킹	36.5

◆ 결과 분석:

- Seq2Seq 모델이 기존 SMT 번역 모델을 초월하는 성능을 보였음.
- 특히 LSTM이 문장의 길이가 길어질수록 성능이 좋아지는 경향을 보였음.
- 순서를 뒤집은 데이터(reverse input sentences)를 사용하면 성능이 **25.9 → 30.6 BLEU**로 대폭 상승.

기술적 분석

(1) LSTM 인코더-디코더 구조

- **입력(Encoder):** 문장을 한 단어씩 LSTM이 처리하며 마지막 타임스텝에서 **고정 길이 벡터**를 생성.
- **출력(Decoder):** 생성된 벡터를 기반으로 새로운 문장을 생성.

(2) 문장 순서 뒤집기 효과

- 일반적으로, 번역에서는 문장의 처음과 끝 부분이 중요한 정보를 포함.
- 문장을 거꾸로 입력하면 디코더가 번역을 생성할 때 **앞부분의 정보를 쉽게 활용**할 수 있음.
- 즉, 단어 간의 단기 의존성(short-term dependency)이 강화되어 학습이 원활해짐.

(3) 병렬 처리 최적화

- 단일 GPU에서는 **1,700 단어/초** 처리 속도 → 8개 GPU 병렬 처리 후 **6,300 단어/초**로 가속화.
- **4개 GPU는 LSTM 계층을 분할**, 나머지 4개는 Softmax 연산을 수행하여 속도 향상.

5. 결론 및 향후 연구 방향

- **LSTM을 활용한 Seq2Seq 모델**은 단순하지만 강력한 성능을 보여주며, 향후 다양한 시퀀스 학습 문제 (번역, 음성 인식 등)에 적용 가능.
- **입력 문장을 뒤집는 전략**이 단기 의존성을 강화하여 학습을 쉽게 만드는 중요한 트릭임.
- 이후 연구에서는 **Attention Mechanism**을 도입하여, 보다 효과적으로 문장 내 정보에 집중하는 방법을 연구할 필요.

Attention Is All You Need

저자: Ashish Vaswani et al. (Google Brain, Google Research, University of Toronto)

발행: NeurIPS 2017

이 논문에서는 기존 **RNN** 및 **CNN**을 사용한 시퀀스 변환 모델을 대체할 **Transformer** 모델을 제안.

핵심 기여

- 순환(RNN) 및 합성곱(CNN) 없이 **Self-Attention**(자기 주의) 기법만으로 문장 간의 관계를 학습하는 **Transformer 아키텍처**를 제안.
- 기존 RNN 기반 모델보다 병렬처리가 가능하여 학습 속도가 빠르고 성능이 뛰어남.
- **WMT'14** 기계 번역(영어-독일어, 영어-프랑스어) 데이터셋에서 **SOTA** 성능 달성 (BLEU: EN-DE 28.4, EN-FR 41.8).

Transformer 모델 개요

(1) 기존 문제점

- **RNN(LSTM, GRU)**
 - 시퀀스를 한 단계씩 처리하여 병렬 연산이 어려움 → 학습 속도 느림
 - 긴 문장에서 장기 의존성(Long-term dependency) 학습이 어려움.
- **CNN 기반 모델(ConvS2S, ByteNet 등)**
 - 문장 길이에 따라 필요한 연산량 증가 → 멀리 떨어진 단어 간 관계 학습 어려움.

(2) Transformer 모델의 핵심 아이디어

- 완전히 **Self-Attention**을 활용하여 문장 내 단어들의 관계를 학습.
- RNN 없이 완전 병렬 처리 가능 → 학습 속도 증가.
- 기존보다 멀리 떨어진 단어 간 관계를 쉽게 학습할 수 있음.

Transformer 아키텍처 구성

Transformer 모델은 **Encoder-Decoder** 구조를 따름.

(1) Encoder & Decoder 구조

- **Encoder:** 입력 문장을 여러 층의 Self-Attention과 Feed-Forward Neural Network(FFN)를 거쳐 표현 벡터로 변환.
- **Decoder:** Encoder의 출력을 참고하여 번역 문장을 생성.

◆ 각각 6개의 층(Stacked Layers)으로 구성

◆ **Self-Attention + Position-wise Feed-Forward Networks + Residual Connection** 포함

(2) Self-Attention 메커니즘

- 기본 원리: 문장 내 모든 단어 간의 연관성을 학습하는 방식.
- **Scaled Dot-Product Attention** 사용

✓ (3) Multi-Head Attention

- 하나의 Attention만 사용하면 모델이 단순해짐 → 여러 개의 **Attention Head** 사용
- 여러 개의 독립적인 **Self-Attention**을 병렬 수행하여 다양한 의미 표현 학습 가능

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

◆ Self-Attention vs Multi-Head Attention

- **Self-Attention:** 단어 간의 관계 학습
- **Multi-Head Attention:** 여러 시점에서 관계를 학습하여 더 정교한 문맥 이해 가능

(4) Position-wise Feed-Forward Networks (FFN)

- Attention을 통과한 벡터를 개별적으로 처리하는 2개의 Fully Connected Layer.
- 비선형 활성화 함수(ReLU) 사용

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{\text{ff}} = 2048$.

(5) Positional Encoding (위치 정보 추가)

- Self-Attention은 단어 순서를 고려하지 않음 → 따라서 입력에 위치 정보를 추가해야 함.
- Sine & Cosine 함수 기반의 위치 인코딩(Positional Encoding) 적용

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

실험 및 결과

WMT'14 기계 번역 성능 (BLEU 점수)

모델	EN→DE	EN→FR	학습 비용
ByteNet	23.75	-	-
GNMT (Google)	24.6	39.92	1.4×10^{20} FLOPs
ConvS2S	25.16	40.46	9.6×10^{18} FLOPs
Transformer (Base)	27.3	38.1	3.3×10^{18} FLOPs
Transformer (Big)	28.4	41.8	2.3×10^{19} FLOPs

◆ 결과 분석:

- Transformer(Big) 모델이 기존 모든 번역 모델을 초월하는 성능을 보임.
- 학습 속도가 빠르면서도 성능이 뛰어남 → 기존 GNMT보다 4배 빠른 학습 속도.
- 모든 기존 CNN/RNN 기반 모델보다 우수한 BLEU 점수 달성.

5. Transformer vs 기존 모델 비교

특징	RNN (LSTM/GRU)	CNN (ConvS2S)	Transformer
병렬 연산 가능 여부	✗ (X)	✓ (O)	✓ (O)
장기 의존성 학습	✗ (X)	✓ (O)	✓ (O)
학습 속도	느림	빠름	가장 빠름
성능 (BLEU)	낮음	높음	최고